mlp_mnist.py

```
# -*- coding: utf-8 -*-
"""
mlp_mnist.ipynb
"""
#%%
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from torchvision import datasets, transforms

batch_size = 2048
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
train_loader = torch.utils.data.DataLoader(
        datasets.MNIST('data', train=True, download=True,
                       transform=transforms.Compose([
                           transforms.ToTensor(),
                       ])),
        batch_size=batch_size, shuffle=True)

test_loader = torch.utils.data.DataLoader(
        datasets.MNIST('data', train=False, transform=transforms.Compose([
                           transforms.ToTensor(),
                       ])),
        batch_size=batch_size, shuffle=True)

train_loader.dataset.data.shape,

class mlp(nn.Module):
  def __init__(self):
    super(mlp,self).__init__()
    self.l1 = nn.Linear(784,128)
```

```
    self.l2 = nn.Linear(128,10)

  def forward(self,x):
    a1 = self.l1(x)
    x1 = F.relu(a1)
    a2 = self.l2(x1)
    return a2

model = mlp().to(device)
optimizer = optim.SGD(model.parameters(),lr=0.01)
model

epochs = 30
for epoch in range(epochs):
  model.train()
  for batch_idx,(x,y) in enumerate(train_loader):
    x,y = x.view(x.shape[0],-1).to(device),y.to(device)
    optimizer.zero_grad()
    output = model(x)
    loss = F.cross_entropy(output,y)
    loss.backward()
    optimizer.step()

  model.eval()
  correct = 0
  test_loss = 0
  with torch.no_grad():
    for batch_idx,(x,y) in enumerate(test_loader):
      x,y = x.view(x.shape[0],-1).to(device),y.to(device)
      output = model(x)
      test_loss += F.cross_entropy(output,y)
      pred = output.max(1,keepdim=True)[1]
      correct += pred.eq(y.view_as(pred)).sum().item()

    test_loss = test_loss/(batch_idx+1)
    acc = correct/len(test_loader.dataset)
    print('epoch:{}, loss:{:.4f}, acc:{:.4f}'.format(epoch,test_loss,acc))
```